# Milestone 7 Final documentation package

I will first construct the <theory-of-the-program>, then generate <program text> only after the theory is explicit.

### OPERATIONAL_DIAGNOSTIC

**Substantives_Demolished** Replace vague uses of completeness, authenticity and understanding with the actions and records below.

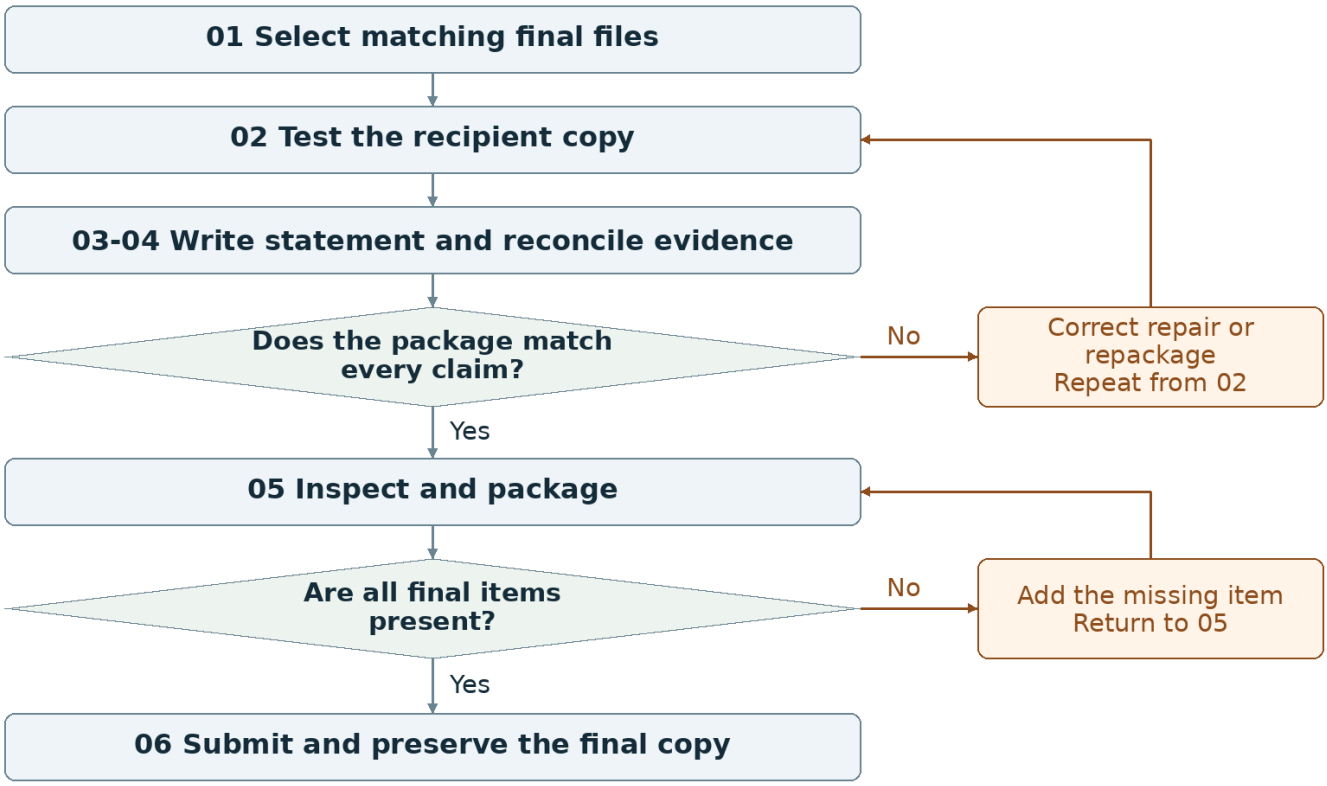
**Occult_States_Removed** Replace the project is finished with an opened final package, a successful documented run and a file-by-file comparison of claims, images and credits.

**The_Language_Game** Keep every submitted claim traceable to the same final version or to a labeled historical record.

### CONTROL_SURFACE_DRAFT

Match the final code, playable copy, statement, images and credits; test the delivery copy and submit the complete record.

**Start with** Gather the presentation build, code, actual feedback, current screenshots, all source credits, AI records and documented limitations from M1 through M6.




Use the numbered procedure; repeat checks only where the repair affects their result.

**Required package** Submit the final code folder, playable build, 300-500-word statement, screenshot set and complete records.

Assign maker _____ and checker _____. Swap roles at the next task. Leave checks pending until performed.

# Milestone 7 Write the rules

### 1 <Initial Interpretation>

Write the task a new recipient will perform: <recipient> [opens] <package>, [follows] <launch instructions>, [uses] <controls>, and [reaches] <outcome>. Document the same version that the recipient will run.

### 2 <Theory Skeleton>

| Record | What to write |
|---|---|
| <Entities> | List final source folder, playable copy, version, statement, screenshots, credits, AI entries and known limitations. |
| [Operations] | Select, launch, compare, trace, count words, reconcile credits and inspect the final upload copy. |
| <States> | Document the actual final screens/states and endings. Remove claims about abandoned states or label them as past work. |
| <Conditions> | Document final input guards and ending tests with code locations. Keep word-count and package checks on the submission checklist. |
| <Invariants> | List final scoped runtime rules with code locations and checks. Keep consistent build identifiers as a separate packaging convention. |
| <Constraints> | Provide title, actual team names, source folder, playable build, statement, screenshot set, full credits and cumulative AI statement. |

Use conditions to decide whether an action or transition is allowed. State the scope of each invariant and how to check it. Keep submission requirements separate from runtime rules.

### 3 <Assumption Ledger>

| Status | Action |
|---|---|
| <safe> | Enter an unverified premise only with its basis and limited consequence if wrong. Keep build-specific results and adopted decisions in the evidence record. |
| <uncertain> | List untested recipient environments or unsupported features; do not generalize successful local checks. |
| <requires-user-decision> | Have both partners select the final version and complete the group confirmation after tracing submitted code and images. |

Create a final unresolved-items list with {item, effect on recipient, evidence, action}. Resolve required omissions before submission; disclose actual remaining limits without inventing tests.

**Record the next open item** Claim _____ Status _____
Owner _____ Evidence/check _____ Next action _____

Write <entity> for a person, object, value or state; [operation] for an action; and {record} for related fields. Record planned, observed, failed or not tested beside each claim. Do not mark a check passed until a person performs it and records the result.

# Milestone 7 Run the procedure

### 4 <Operational Description>

Read each action aloud before doing it. The maker performs the action; the checker records the named result. Use the return instruction when a check fails.

| Step | Perform and save | Check and return |
|---|---|---|
| 01 Select the final version | Match the final source folder to the playable build. Remove abandoned experiments from the delivery folder; preserve a separate working archive if needed. | Locate the source of every submitted behavior. Resolve a mismatch before writing final claims. |
| 02 Test the recipient copy | Open the final delivery copy using its included instructions. Start, use the displayed control and reach an ending. Record environment, network condition and observed result. | Repair packaging or behavior, regenerate the final copy and repeat the affected checks. Record untested platforms honestly. |
| 03 Write the statement | Write 300-500 words in four paragraphs: project; WarGames connection; code structure/mechanics/design process; AI use. Add title and actual team names outside that body. | Count the statement body and check all four topics. Edit claims that describe absent features or tests that were never performed. |
| 04 Reconcile the evidence | Use screenshots from the final build. Match borrowed assets, code, tutorials, fonts, sounds, inspirations and AI assistance to credits and code comments. | Correct missing sources, obsolete screenshots and inconsistent AI dates or kept/changed/rejected entries. |
| 05 Inspect and package | Each partner traces a code change and identifies image sources, edits and displayed information; then complete the group confirmation. Include code, playable build, statement, screenshots, full credits, cumulative AI record and final progress note. | Open the exact final files. Leave the confirmation pending until both partners perform the recommended checks and can confirm the submitted work. |
| 06 Submit and preserve | Use the actual Canvas upload fields. After upload, inspect the listed files and reopen or download them when available. Retain the submitted copy and receipt. | Record the submitted names and status. Do not mark submission complete merely because files exist locally. |

### 5 <Failure Description>

| Observed problem | Required response |
|---|---|
| Statement and build describe different behavior | Correct the text or deliberately repair the behavior, then repeat the relevant check. |
| Word count or required topic is wrong | Edit the four-paragraph body in step 03; recount and check every topic. |
| Missing credit or partner confirmation | Resolve the specific source or demonstration in steps 04-05. Do not sign for the other partner. |

**Use the map** A no branch returns to the named task. Repair the recorded problem and repeat that check; do not mark the stage passed because the planned time window ended.

# Milestone 7 Test changes and produce the work

### 6 <Change Test>

Ask: "What would change if the requirements changed?" The maker writes the expected result first. The checker performs or traces the change and records the result. Swap roles for the second scenario.

| Modification | Run this check |
|---|---|
| Change a rule parameter | In a separate test copy, choose one parameter. Predict the affected behavior, files, instructions and checks; change it and compare the run. Restore or deliberately adopt the change and update all affected records. |
| Replace the visual treatment | Identify which colors, shapes or text carry information and which are decorative. Propose a style replacement. Verify that controls, state distinctions and outcome information remain readable and distinguishable. |

Save {version or design record, starting condition, input/change, expected result, observed result, pass/fail/not tested, next action}. Use paper traces in M1-M2; report executed checks only when they occurred.

### 7 <Implementation Plan>

Assign the first statement draft and file inventory to one partner, the final launch and credit reconciliation to the other, then swap review. Keep supporting technical notes outside the 300-500-word statement. Use four paragraphs to cover the four required topics.

### 8 <Program Text>

Produce final documentation and packaging. Change implementation only for a named defect; write the intended behavior first, apply a bounded repair and repeat affected checks. Update the final identifier and matching evidence afterward.

### 9 <Theory-Code Mapping>

| Element | Record the connection |
|---|---|
| <types> | Document final data names, meaning, valid values and reset values. |
| <functions> | Map input, rule updates, drawing and outcomes to actual file/function names. |
| <classes> | List actual responsibilities or mark not used. |
| <tests> | Retain final launch, input, outcome and modification records. |
| <comments> | Preserve non-obvious reasons, limitations and AI assistance annotations. |
| <configuration> | Name safe-to-change values and the checks required after each change. |

Use actual file and function names when code exists. Label planned counterparts proposed; write not applicable when a structure is absent.

### Carry forward the course rules

Keep the project connected to WarGames. Build in Processing using p5.js mode; accept interactive input; include at least three distinct screens or states; provide a start and reachable outcome; credit external material; organize and comment the code.

Record actual AI use at every milestone, including use of this manual if adopted. Use AI for bounded brainstorming, explanation, small examples you study and adapt, specific debugging or review. Author the project as a group; do not request an entire generated project. Identify assisted code and your changes in comments.

# Milestone 7 Submit and hand off

### 10 <Residual Human Theory>

Record why the group selected its mechanic, interpreted peer observations and rejected alternatives. Explain which rule changes require a new diagram or playtest. Have each partner locate those records and demonstrate one small change before completing the group confirmation.

### Check the required submission

- Final code folder and final playable build.
- A 300-500-word project statement covering project, WarGames connection, code/mechanics/process and AI use.
- Project title and actual student/team names.
- A final screenshot set and complete credits/references.
- An AI Use Statement covering the whole project and a short final progress note.
- Optional short video or GIF; keep the required playable build regardless.

Organize code, playable build, statement, screenshots and records into clearly named items. Follow current Canvas fields; the supplied final-package instructions set no PDF minimum or explicit ZIP requirement.

**Completion check** The checker runs the final delivery copy by its documented route, verifies the 300-500-word body and four topics, and opens all required files. Both partners complete the group confirmation; the upload record identifies the delivered copy.

**Progress note** Name the current version; state what was completed and checked, what is missing, and the next action with its owner. Add the screenshot/capture supporting the current status.

### Complete the AI Use Statement

| Field | Enter the actual record |
|---|---|
| Tool and dates | Name the tool, or record no AI used only if accurate. Enter actual dates of use. |
| Request and affected work | Summarize the request. Name the text, image, planning record or code file affected. |
| Kept changed rejected | Record the output retained, edits or rewrites made, and material discarded. |
| Group confirmation | Recommended check: each partner traces actual submitted code; for planning-only work, trace the proposed interaction. Identify image sources, retained edits and displayed information. Then complete the required group confirmation covering code and images; leave it pending until both partners can confirm it. |

If this AI-produced manual is adopted, record ChatGPT, October 21 2026, milestone planning and operational rewrite, and your actual kept/changed/rejected choices. Do not copy a completed student confirmation from this manual.

**Handoff** Preserve the exact submitted version, launch instructions, check evidence and reasons for the rules so a later editor can locate and verify a proposed change.

Record: maker _____ checker _____ version _____
Checks still pending _____ submitted file(s) _____ upload status _____

Requirements source: supplied WarGames project brief and milestone instructions. Treat roles, file names, internal checks and relative-week allocations as recommended working procedures. Use corrected deadlines from the course; none are asserted here.